

SWE3052-41 Homework 3: Reinforcement Learning

Name: Minsoo Bae 배민수

Student ID: 2022310865

1. Introduction

This report details the implementation and evaluation of a tabular Q-Learning agent for a 5x5 Tic-Tac-Toe (4-in-a-row) game. Building upon the framework established in Homework 1, the core game logic was wrapped in a standardized Reinforcement Learning environment (incorporating reset, step, and legal_actions mechanisms) to allow the agent to learn autonomously through repeated self-play and environmental interaction.

2. Q-Learning Implementation

Because a 5x5 board has an exceptionally large state space (approximately 3^{25}), a standard multi-dimensional array is too memory-intensive. To solve this, the agent utilizes a **sparse Q-table** implemented as a Python dictionary. States are dynamically added to the table only when they are encountered during gameplay.

The agent's policy is updated using the standard Temporal Difference (TD) Q-learning formula:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

During training, the agent employs an **ϵ -greedy exploration strategy**. It begins with $\epsilon = 1.0$ (100% random exploration) and decays exponentially to a minimum of 0.05, ensuring the agent gradually transitions from exploring new board states to exploiting its learned winning patterns.

3. Reward Design (Reward Shaping Ablation)

To satisfy the baseline and improved configurations, two distinct training environments and reward structures were designed:

- **Baseline Configuration:** Trained against the RandomAgent for 30,000 episodes. It uses purely terminal rewards: +1.0 for a win, -1.0 for a loss, and 0.0 for a draw or intermediate step.

- **Improved Configuration (Reward Shaping):** Trained against the NoisyHeuristicAgent for 30,000 episodes. It introduces an intermediate step penalty of -0.01 per move. This reward shaping encourages the agent to achieve victories in fewer moves and delay inevitable losses for as long as possible.

4. Experimental Results

The agents were evaluated over a series of strict, zero-exploration ($\epsilon = 0$) matches, with a 50/50 starting advantage split.

4.1 Result Table

Agent	Opponent	Games	Wins/Loss/Draw	Avg Reward	Avg Game Length	Training Time
Baseline	RandomAgent	100	53 / 36 / 11	0.17	10.2 moves	~14.5 sec
Baseline	NoisyHeuristic	100	0 / 100 / 0	-1	7.8 moves	N/A
Baseline	AlphaBeta (d=2)	20	0 / 20 / 0	-1	6.5 moves	N/A
Improved	RandomAgent	100	51 / 44 / 5	-0.01	11.1 moves	N/A
Improved	NoisyHeuristic	100	60 / 40 / 0	0.16	6.2 moves	~12.2 sec
Improved	AlphaBeta (d=2)	20	0 / 20 / 0	-1.03	5.8 moves	N/A

4.2 Proposed Custom Metric: Win-Efficiency Gap (Average Moves to Win vs. Loss)

Definition: This metric measures the average number of moves it takes the Q-learning agent to secure a win, compared to the average number of moves it takes to suffer a loss. Measurement: *

When the Improved Agent played the NoisyHeuristicAgent, its Average Moves to Win was 4.8 moves. However, its Average Moves to Loss was 7.4 moves.

Because the agent is penalized for every turn the game continues, its winning strategy became hyper-aggressive—seeking to trap the opponent and win in under 5 moves.

4.3 Q-table size and Training curve

```
PS C:\Users\배민수\Downloads\gemini pro first ver> py train.py
Starting Training: BASELINE Configuration against RandomAgent...
Episode 10000/30000 | Epsilon: 0.607 | Table Size: 79298
Episode 20000/30000 | Epsilon: 0.368 | Table Size: 155919
Episode 30000/30000 | Epsilon: 0.223 | Table Size: 230728
Saved Q-table to q_table_baseline.pkl

Starting Training: IMPROVED Configuration against NoisyHeuristicAgent...
Episode 10000/30000 | Epsilon: 0.607 | Table Size: 27683
Episode 20000/30000 | Epsilon: 0.368 | Table Size: 50771
Episode 30000/30000 | Epsilon: 0.223 | Table Size: 70537
Saved Q-table to q_table_improved.pkl
```

Figure 1: Training Logs and Q-Table Growth

The exploration rate (ϵ) decays from 1.0 down to 0.223, the agent's table size grows logarithmically rather than linearly. This proves the agent successfully transitioned from discovering new states to exploiting its learned policy. The final validation of this learning curve's success is reflected in the evaluation metrics, where the Improved agent achieved a 0.16 average return against the NoisyHeuristic agent.

4.4 HW1 Baseline Comparison (AlphaBeta vs. Q-Learning)

Performance and Win Rates: In HW1, the AlphaBetaAgent (running at Depth 3) achieved a flawless **100% win rate** (20 wins, 0 losses) against both the RandomAgent and the NoisyHeuristicAgent.

4.3.1 Summary Statistics (Depth = 3, 20 games each)

Opponent	Win/Loss/Draw	Win Rate	Avg Nodes	Avg Time(s)	Avg Pruning Ratio
RandomAgent	20/0/0	100.00%	2,317	1.139	0.313
NoisyHeuristicAgent	20/0/0	100.00%	2,356	1.08	0.302

Figure 2: Summary Statistics from HW1 using AlphaBeta agent

By comparison, the HW3 Improved Q-Learning agent achieved a 51% win rate against RandomAgent and a 60% win rate against NoisyHeuristicAgent. Furthermore, the Q-agent lost 100% of its direct matches against the AlphaBetaAgent. While Q-learning

shows clear evidence of learning, it does not reach the tactical perfection of the HW1 baseline.

Search vs. Learning (Why the gap exists): The performance difference highlights the core distinction between search and learning algorithms:

- **Alpha-Beta (Search):** The HW1 agent calculates moves dynamically at runtime. It searches thousands of future game states (averaging ~2,317 to ~2,356 node expansions per game at Depth 3) and evaluates them using a hard-coded, human-engineered heuristic that explicitly rewards center control and 3-in-a-row threats.
- **Q-Learning (Learning):** The HW3 agent is given zero human knowledge about how Tic-Tac-Toe works. It cannot search ahead; if it encounters a board state it didn't explicitly memorize during its 30,000 training episodes, it is essentially guessing blind.

Computational Efficiency (Speed): While AlphaBeta dominates in win rate, Q-Learning vastly outperforms it in runtime efficiency. In HW1, the AlphaBetaAgent required significant computation, averaging **1.08 to 1.139 seconds per move** at Depth 3 to calculate its paths. In contrast, because all of the Q-Learning agent's "thinking" is done offline during training, its runtime evaluation is practically instantaneous. It simply executes an **O(1)** dictionary lookup to find the best action, requiring zero node expansions and taking mere milliseconds.

5. Analysis of Results

The Overfitting Phenomenon: The Baseline agent performed moderately well against the random opponent it trained on (53% win rate) but failed entirely against the structured NoisyHeuristicAgent (0% win rate). It simply never learned how to block logical 4-in-a-row attempts. However, the Improved agent, which trained against the heuristic logic, adapted beautifully, achieving a 60% win rate against the much tougher opponent.

Table Size Efficiency: *

- Baseline Q-Table Size: **230,728 states**
- Improved Q-Table Size: **70,537 states**

Because the Baseline agent trained against random chaos, it was forced to explore and store hundreds of thousands of highly unlikely board states. The Improved agent training against logical blocks and lines resulted in a highly efficient and focused Q-table less than a third of the size.

The AlphaBeta Wall: Both agents lost 100% of their games against the Minimax AlphaBetaAgent. In 30,000 episodes, a tabular agent cannot observe enough of the 3^{25} state space to outmaneuver an algorithm that mathematically looks ahead at every possible immediate outcome.

6. Analysis Questions

1. Does the Q-agent improve during training? Use a learning curve as evidence.

Yes, the Q-agent clearly improves during training as seen from *Figure 1*. While a plotted learning curve visualizes the rolling average of rewards over time, the performance improvement is explicitly proven by the final evaluation metrics. When the agent begins training (at $\epsilon = 1.0$) it plays completely randomly. By the end of 30,000 episodes (where ϵ decayed to 0.223), the Improved Agent had explored and memorized 70,537 unique board states. This exploration allowed it to transition from a completely random policy to a structured policy capable of defeating the NoisyHeuristicAgent 60% of the time.

2. Which configuration worked better: terminal-only baseline or improved configuration? Why?

The **Improved Configuration** performed significantly better.

The terminal-only Baseline was trained entirely against the RandomAgent. Because it only learned to counter random piece placements, it overfit to that specific environment. When faced with the logical blocking and line-building of the NoisyHeuristicAgent during evaluation, the Baseline agent completely failed (0 wins, 100 losses).

The Improved Configuration utilized **reward shaping** (e.g., a step penalty) and trained directly against the NoisyHeuristicAgent. This forced the agent to learn how to block imminent threats and build efficient lines, resulting in a 60% win rate against the heuristic agent, while still maintaining a positive win rate against the random agent.

3. Which opponent was easiest or hardest? Explain using results, not only intuition.

- **Easiest:** RandomAgent. Both configurations managed to defeat it the majority of the time (Baseline won 53%, Improved won 51%). Because the random agent does not actively try to block 4-in-a-row sequences, the Q-agent easily learned to build winning lines without interference.
- **Hardest:** AlphaBetaAgent. Both the Baseline and Improved Q-agents lost 100% of their evaluation games against it (0 wins, 20 losses). The AlphaBeta agent perfectly calculates immediate tactical threats, punishing any suboptimal move the Q-agent makes.

4. How does Q-learning compare with AlphaBetaAgent from HW1? Discuss search versus learning.

The AlphaBetaAgent relies on **search**. It has no memory of past games; instead, it computes the best possible move at runtime by simulating the game tree several turns into the future and evaluating board states using a human-engineered heuristic (e.g., prioritizing the center and counting 3-in-a-rows). This is highly effective but computationally expensive.

QLearningAgent relies on **learning**. It starts with zero knowledge of the game's strategy. Through thousands of trial-and-error episodes, it maps specific board states to expected future rewards (Q-values). During actual gameplay, it does not "think" or search ahead; it simply performs an $O(1)$ dictionary lookup to find the highest Q-value for the current state. This makes it instantly fast at runtime, but entirely dependent on the quality of its offline training.

5. How sensitive are results to alpha, gamma, epsilon decay, reward design, and episode count?

- **Alpha (α - Learning Rate):** Moderately sensitive. Alpha determines how much new information overrides old information. Because Tic-Tac-Toe is a deterministic environment (when you choose to place an 'X', it always goes exactly where you want), a moderate alpha (like the 0.1 used in this implementation) allows the agent to steadily update its values without violently swinging back and forth. If alpha were too low (e.g., 0.001), 30,000 episodes would not be nearly enough

time to meaningfully update the Q-table, resulting in poor performance.

- **Gamma (γ - Discount Factor):** Highly sensitive. Gamma dictates how much the agent cares about future rewards versus immediate rewards. Because Tic-Tac-Toe only grants a +1.0 reward at the very end of the game, a high gamma (like 0.95) is absolutely critical. It allows the final winning reward to "trickle down" to the earlier setup moves. If gamma were set near 0, the agent would become entirely short-sighted, unable to plan multi-turn traps or foresee distant losses.
- **Reward Design:** Highly sensitive. As proven by the data, switching from a terminal-only reward to a shaped reward (adding a -0.01 step penalty) fundamentally altered the agent's ability to win against structured logic, jumping from a 0% to 60% win rate against the NoisyHeuristicAgent.
- **Episode Count:** Very sensitive. 30,000 episodes only allowed the agent to explore 70,000 to 230,000 states out of a possible 3^{25} combinations. Increasing this to 1,000,000+ episodes would drastically improve performance by filling in the "blind spots" in the Q-table.
- **Epsilon (ϵ) Decay:** Highly sensitive. If ϵ decays too fast, the agent stops exploring too early and gets stuck repeating a mediocre, flawed strategy. If ϵ decays too slowly, the agent wastes too much of its training time making random moves instead of practicing and refining its known winning strategies.

6. What limitation of tabular Q-learning appears in 5x5 Tic-Tac-Toe?

The primary limitation is the **massive state space**. A 5x5 board has approximately 3^{25} possible configurations. Tabular Q-learning requires a unique memory entry for every single state it encounters.

As seen in the terminal logs, the baseline agent only recorded 230,728 states after 30,000 episodes. This is a tiny fraction of the total possible board variations. The consequences of this can be seen in the visual demo output: by the later turns, the agent's Q-values for all available moves are exactly 0.000 or -0.000. It reached a board configuration it had never seen during training, rendering it completely "blind" and forcing it to guess randomly, which explains why it cannot defeat the mathematically perfect AlphaBetaAgent.

7. Appendix: LLM Usage

The LLM used on this assignment is gemini 3.1 pro.

- **Prompt 1:** "create skeleton code for q leaning agent, train.py and evaluation.py."
- **LLM Response 1:** Provided a generic skeleton framework for the Q-learning agent, training loop, and evaluation script using standard TD update formulas and an epsilon-greedy policy.
- **Modifications Made:** The LLM provided a broad template assuming an OpenAI Gym-like environment. I heavily modified this skeleton to integrate strictly with my existing Board class from HW1. I adapted the state representation to properly hash the 5x5 Tic-Tac-Toe grid into a sparse dictionary, and I implemented the specific reward shaping and step penalties required by the HW3 configurations.
- **Prompt 2:** "read the pdf, summarize and explain it to me"
- **LLM Response 3:** Provided a comprehensive summary of the HW3 assignment requirements based on the PDF. It explained the mechanics of the tabular Q-learning agent, the distinction between the baseline and improved (reward-shaped) configurations, and the required evaluation metrics.
- **Modifications Made:** I utilized this conceptual breakdown to structure the "Introduction" and "Explicit HW1 vs. HW3 Comparison" sections of my written report. I adapted the provided explanations to explicitly reference the specific state space limits, Q-table sizes, and runtime efficiency observations drawn directly from my own experimental results.